

A Secure OCC Testbed for Industrial Protocol Benchmarking and Cyber-Attack Evaluation

Moulikha Mahendran

NETWORK ARCHITECTURE: CLIENT-SERVER DICHOTOMY

The architecture relies on two distinct server roles to isolate local operations from global analytics.

1. The Edge (Local Client-Server)

Environment: The isolated, physical testing ground.

- **Hardware Role:** Raspberry Pi acts as the **Local Server** (Broker/Middleware).
- **Client Role:** ESP32 units act as Autonomous Vehicle **Clients**.
- **Network:** Closed Local Area Network (LAN) / Dedicated Wi-Fi Hotspot.
- **Local IP Range:** `192.168.1.0/24` subnet.
- **Function:** Immediate data ingestion, initial protocol parsing, and real-time Threat Detection processing.

2. The OCC (Main Server)

Environment: The Operations Control Center.

- **Hardware Role:** Remote PC / Cloud Dashboard acts as the **Main Server**.
- **Client Role:** The Raspberry Pi acts as a **Client** to this Main Server.
- **Network:** Wide Area Network (WAN) / Secure VPN.
- **Remote IP Range:** e.g., `203.0.113.10` (Public/Routed).
- **Function:** Aggregation of safe data, KPI calculation, long-term persistence, and Grafana Dashboard rendering.

LAYER 1: AUTONOMOUS VEHICLE LAYER (CLIENTS)

Hardware & Software Build

The perception layer consists of ESP32 microcontrollers tasked with generating telemetry data, acting as simulated Autonomous Guided Vehicles (AGVs).

- **Software Stack:** MicroPython or Arduino C++ using core libraries (WiFi.h, PubSubClient.h).
- **Payload Generation:** Custom scripts simulating Speed, Position (x,y), Battery Life, and Task Status.
- **Data Format:** Standardized JSON payloads with integrated checksums and strict packet sequencing.

IP Addressing & Subnet Strategy

Device Profile	Assigned IP	Role
Vehicle 1 (Pi)	192.168.1.10	Telemetry Gen
Vehicle 2 (ESP32)	172.20.10.5	Station Mode
Vehicle 3 (ESP32)	10.0.0.15	Alt Subnet

Traffic Mode Selection: Devices are programmed to execute either a **Normal Simulation** (standard intervals) or an **Attack Simulation** (malformed data or flooding) prior to publishing.

LAYER 2: COMMUNICATION PROTOCOL IMPLEMENTATION

This layer represents the core variable of the testbed. We actively switch between protocols to benchmark performance across different transport methods (Wi-Fi, LAN).

MQTT / TLS

- **Model:** Publish / Subscribe.
- **Core Service:** Eclipse Mosquitto Broker at `192.168.0.20`.
- **Security:** TLS Encryption, Username/Password Auth.
- **Testing:** QoS 0, 1, and 2 latency differences.

OPC UA

- **Model:** Server / Client architecture.
- **Core Service:** FreeOpcUa / Python opcua.
- **Security:** Secure Channel, X.509 Certificate validation.
- **Testing:** Read/Write Data Model processing overhead.

DDS & VDA 5050

- **DDS Model:** RTPS over UDP. Tests Access Control and Encryption.
- **VDA 5050 Model:** JSON over MQTT/HTTP. Tests standard AGV order/state communication and specific topic handling.

LAYER 3: CYBERSECURITY MIDDLEWARE (THE CORE)

Hosted on the Raspberry Pi (64-bit OS), this layer executes Python/Node-RED logic to process data before it reaches the Main Server. It relies on 5 sequential modules.

1. Data Ingestion

Receives multi-protocol data. Decodes and parses incoming messages.

2. Protocol Handler

Routes traffic based on origin (MQTT Handler, OPC UA Handler, DDS Handler).

3. Data Validation

Performs Schema Checks, Range Checks, and Timestamp Validation.

4. Threat Detection Engine

Applies advanced logic to the validated data: Rule-based detection, strict Threshold limits, Anomaly recognition, and Signature matching for known attack vectors.

5. Traffic Monitoring

Runs parallel packet capture (Wireshark/Tcpdump), flow analysis, and behavior monitoring to extract physical network metrics.

LAYER 3: ATTACK HANDLING & DECISION LOGIC

The Threat Detection Engine dynamically alters the data flow model based on real-time analysis. If an anomaly is detected, normal forwarding is suspended.

Branch A: Anomaly Detected

Traffic is instantly diverted to the **Attack Handling Module**.

- **Action 1:** Log attack details to local Attack Database (Type, Source, Time, KPI Impact).
- **Action 2:** Classify the attack severity.
- **Action 3:** Generate immediate local alert.
- **Action 4:** Quarantine node or Block/Drop malicious traffic via UFW/iptables.

Branch B: Normal Operation

Traffic is verified safe and proceeds to the **KPI Engine**.

- **Action 1:** Data is officially approved.
- **Action 2:** KPI Engine measures exact Latency, Throughput, Jitter, and Packet Loss for this specific transmission.
- **Action 3:** Clean, verified data is packaged and forwarded out of the local LAN to the OCC Dashboard.

LAYER 4: OCC DASHBOARD LAYER (MAIN SERVER)

The Main Server (Cloud or Isolated PC) utilizes tools like Grafana and Plotly/Dash to visualize the data processed by the Edge Middleware.

KPI Visualization

- Real-time graphs plotting Edge latency vs. Main Server latency.
- Protocol Comparison (e.g., MQTT vs OPC UA speed side-by-side).
- Protocol-wise KPI dashboards.

Attack Alerts

- Visual Alarm panels triggering on Middleware flags.
- Identification of Attack Type (DDoS, Replay).
- Source/Time tracking and severity scoring.

System Status & Reports

- Live Vehicle/Node Status (Active vs. Quarantined).
- Network State and active Protocol tracking.
- Automated CSV/PDF Export for post-simulation performance review.

ATTACK SIMULATION MATRIX (THE STRESS TESTS)

ID	Attack Type	Description	Method / Software Tool
AS-01	DoS / Flooding	High traffic flooding targeting the Mosquitto Broker to saturate CPU.	hping3 / scapy
AS-02	MITM Attack	Man-in-the-middle interception between ESP32 and Gateway.	Bettercap / Ettercap
AS-03	Fake Vehicle	Unauthorized node attempting to publish to core topics.	Custom Python Script
AS-04	Packet Replay	Capturing and re-sending old, valid telemetry packets.	Scapy
AS-05	Packet Tamper	Modifying payloads (e.g., altering speed data) mid-transit.	Scapy

KPI METRICS MATRIX

The success of a protocol under attack is quantified purely by these metrics, calculated locally at the Middleware and visualized at the OCC.

Metric	Description & Evaluation Criteria
Latency (ms)	End-to-end delay. Measures how long a protocol takes to deliver a message under DoS stress compared to its normal baseline.
Throughput (msgs/s)	Messages processed per second. Highlights bandwidth efficiency of MQTT vs OPC UA.
Packet Loss (%)	Percentage of lost packets. Critical for evaluating reliability during network flooding.
Jitter (ms)	Variation in latency. Determines the stability of the protocol connection.
CPU / Memory (%)	Middleware resource usage. Identifies if the security layer (TLS) crashes the Raspberry Pi processor under heavy load.

A TO Z IMPLEMENTATION WORKFLOW

Structured, phase-based execution from bare-metal setup to final deployment testing.

Env & Setup

1

Install OS on Raspberry Pi & ESP32. Configure Wi-Fi/LAN subnets. Setup exact Time Synchronization.

Simulations

2

Generate synthetic telemetry in Python/C++. Implement normal vs. attack states. Validate JSON format.

Protocols

3

Setup MQTT/TLS Broker. Deploy OPC UA Server/Client. Implement DDS/VDA 5050 messaging tests.

Middleware

4

Build Data Ingestion and Validation modules. Script the Threat Detection and KPI Calculation engines.

Attack Execution

5

Run Scapy/hping3 attacks on testbed. Log data, evaluate detection accuracy, and fine-tune threshold rules.

OCC Dashboard

6

Build Grafana UI. Add Real-time KPI graphs, Alert systems, and automated CSV/PDF report extraction.

Final Deployment

7

Run End-to-End performance testing. Perform security hardening. Finalize documentation and presentations.